

МІНІСТЕРСТВО ОСВІТИ І НАУКИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА ФРАНКА
Факультет прикладної математики та інформатики
Кафедра програмування

Лабораторна робота №12
Інтерпретація за формулами; операторне виконання

Виконала:
Студентка групи ПМІ-46
Тригуб Марта

Мета роботи

Вивчити теоретичні основи та приклад інтерпретатора арифметичних формул за алгебраїчними правилами. Розширити базовий інтерпретатор новими бінарними операціями ($\div$, $\%$), унарними операціями, піднесенням до степеня ($**$), математичними функціями ($\sin()$, $\cos()$), а також підтримкою дійсних чисел у форматі фіксованої крапки та в експоненціальній формі. Реалізувати операторне виконання операторів присвоєння та `print()` мінімізованої мови Python.

Теоретичні відомості

Метод рекурсивного спуску

Синтаксичний аналіз формули виконується методом рекурсивного спуску (recursive descent parsing). Для кожного нетермінального символу грамматики будується окрема функція, яка розпізнає праву частину відповідного граматичного правила. Вхідними даними є список лексем, побудований сканером під час першого перегляду тексту.

Розширена граматики виразів

Початкову граматику доповнено новими правилами:

```
----- розширене синтаксичне визначення формули -----
arith_expr ::= term ( ( "+" | "-" ) term ) *
term       ::= factor ( ( "*" | "/" | "//" | "%" ) factor ) *
factor     ::= [ ( "+" | "-" ) ] ( number | "(" arith_expr ")" | function )
            | [ ( "***" factor ) * ]
function   ::= "sin(" arith_expr ")" | "cos(" arith_expr ")"
number     ::= cipher cipher* [ "." cipher cipher* ]
            | [ ("e"|"E") [ "+" | "-" ] cipher cipher* ]
```

Пріоритети операцій (від найвищого до найнижчого): унарні $+$ $-$ застосовуються після $**$; $** > * / // \% > + -$ (бінарні). Операція $**$ є правоасоціативною — реалізується через рекурсивний виклик методу `factor()`.

Граматики операторного виконання

Для мінімізованої $\sim$ Python-мови введена граматики операторів:

```

----- граматика мінімізованої ~Python-мови -----
prog      ::= stmt *
stmt      ::= assign | print_stmt
assign    ::= varname "=" arith_expr
print_stmt ::= "print" "(" arith_expr ")"
varname   ::= letter ( letter | digit | "_" ) *

```

Опис програмної реалізації

arithexpr_extended.py

Сканер (метод scanner()). Сканер виконує перший перегляд тексту формули і будує список лексем у вигляді пар (код, значення). Розпізнавання двосимвольних операторів реалізовано заглядаванням на один символ вперед:

```

def scanner(self):
    """Розбити текст на лексеми; повертає True або False при помилці."""
    while self.i < len(self.text):
        c = self.text[self.i]
        if c == '(': self.leks.append((self.openbracket, '('))
        elif c == ')': self.leks.append((self.closebracket, ')'))
        elif c == '+': self.leks.append((self.add, '+'))
        elif c == '-': self.leks.append((self.subtract, '-'))
        elif c == '%': self.leks.append((self.modulo, '%'))
        elif c == '*':
            # ** або *
            if self.i + 1 < len(self.text) and self.text[self.i + 1] == '*':
                self.leks.append((self.power, '**'))
                self.i += 1          # пропустити другий *
            else:
                self.leks.append((self.multiply, '*'))
        elif c == '/':
            # // або /
            if self.i + 1 < len(self.text) and self.text[self.i + 1] == '/':
                self.leks.append((self.floordiv, '//'))
                self.i += 1          # пропустити другий /
            else:

```

Метод openumber(). Розширений для підтримки дійсних чисел – зчитує цілу частину, необов'язкову дробову (після крапки) та необов'язковий показник степеня (e/E):

```

def onenumber(self):
    """Читати ціле або дійсне число (фіксована крапка / Е-форма). (5.5)"""
    num = ""
    # Цілова частина
    while self.i < len(self.text) and self.text[self.i].isdigit():
        num += self.text[self.i]; self.i += 1
    # Дробова частина
    if self.i < len(self.text) and self.text[self.i] == '.':
        num += '.'; self.i += 1
        while self.i < len(self.text) and self.text[self.i].isdigit():
            num += self.text[self.i]; self.i += 1
    # Показник степеня (е або Е)
    if self.i < len(self.text) and self.text[self.i] in ('e', 'E'):
        num += self.text[self.i]; self.i += 1
        if self.i < len(self.text) and self.text[self.i] in ('+', '-'):
            num += self.text[self.i]; self.i += 1
        while self.i < len(self.text) and self.text[self.i].isdigit():
            num += self.text[self.i]; self.i += 1
    if num:
        val = float(num) if ('.' in num or 'e' in num or 'E' in num) else int(num)
        self.leks.append((self.number, val))

```

Метод funcname(). Виділяє ім'я функції. Підтримуються sin та cos:

```

def funcname(self):
    """Читати ім'я функції sin або cos. (5.3)"""
    name = ""
    while self.i < len(self.text) and self.text[self.i].isalpha():
        name += self.text[self.i]; self.i += 1
    if name == 'sin': self.leks.append((self.funcsin, 'sin'))
    elif name == 'cos': self.leks.append((self.funccos, 'cos'))
    else: return False # невідома функція
    return True

```

Метод term(). Розширений операціями // і %:

```

def term(self):
    """term ::= factor ( ( "*" | "/" | "//" | "%" ) factor ) * (5.1)"""
    z = self.factor()
    while self.leks[self.k][0] in (self.multiply, self.divide,
                                   self.floordiv, self.modulo):
        opr = self.leks[self.k][0]
        self.GetNextToken()
        f = self.factor()
        if opr == self.multiply: z = z * f
        elif opr == self.divide: z = z / f
        elif opr == self.floordiv: z = z // f
        else: z = z % f
    return z

```

Метод factor(). Повністю перероблений: підтримує унарні операції, піднесення до степеня та виклик функцій. Ключова частина:

```

# --- Унарна операція (5.4) ---
unary = None
if self.leks[self.k][0] == self.add:
    unary = self.add; self.GetNextToken()
elif self.leks[self.k][0] == self.subtract:
    unary = self.subtract; self.GetNextToken()

# --- База ---
code = self.leks[self.k][0]
if code == self.number:                                # просте число
    self.GetNextToken()
    val = self.leks[self.k - 1][1]

elif code == self.openbracket:                          # ( arith_expr )
    self.GetNextToken()
    val = self.arithexpr()
    if self.leks[self.k][0] == self.closebracket:
        self.GetNextToken()
    else:
        raise errorexpr

```

arithexpr_stmt.py

Для операторного виконання визначено два класи:

- ExprInterpreter – розширений інтерпретатор виразів з підтримкою змінних (код лексеми varname = 2); при зустрічі ідентифікатора здійснює пошук в словнику variables.
- StmtInterpreter – виконує програму як послідовність рядків; кожен рядок класифікує як оператор присвоєння або print().

Класифікація операторів (метод _exec_stmt()):

```
def _exec_stmt(self, lineno, line):
    stripped = line.replace(' ', '')

    if stripped.startswith('print(') and stripped.endswith(')'):
        return self._exec_print(line, stripped)

    if '=' in line:
        return self._exec_assign(line)

    return False, f"Невідомий оператор: {line}"
```

Присвоювання (_exec_assign()): ліва частина до "=" – ім'я змінної, права – вираз; результат зберігається в словнику self.variables.
print (_exec_print()): вирізає аргумент між print(i), обчислює через ExprInterpreter, виводить на екран.

Тестування

Тести розширеного інтерпретатора виразів

Формула	Результат	Перевірка
49 - 108 / (6+11) * (100-94)	10.882...	базовий тест
17 // 5	3	ділення на ціло (5.1)
17 % 5	2	остача від ділення (5.1)
2 ** 10	1024	ступінь (5.2)
2 ** 3 ** 2	512	правоасоціативність **
sin(0)	0.0	функція sin (5.3)
cos(0)	1.0	функція cos (5.3)
$\sin^2(3) + \cos^2(3)$	≈ 1.0	тригонометрична тотожність

$-2 + -6$	-8	унарний мінус (5.4)
$4 * -2$	-8	унарний після бінарного
$-3 ** 2$	-9	пріоритет: $**$ > унарний -
$45.02 + 0.98$	46.0	фіксована крапка (5.5)
$1.0e-5 * 1e5$	1.0	Е-форма числа (5.5)
$-1.5e1 + 17//3 * 2$	-5.0	комбінований тест

Тест операторного виконання (Програма 1)

```

PROG1 = """
# Програма 1  базова арифметика з присвоюванням i виводом
x = 10
y = 3
q = x // y
r = x % y
print(q)
print(r)
print(x * y + 2)
sum_sq = x ** 2 + y ** 2
print(sum_sq)
"""

```

```

=====
ПРОГРАМА 1
=====
3
1
32
109

```

Тест операторного виконання (Програма 2)

```

PROG2 = """
# Програма 2   дійсні числа, функції, унарні операції
a = 2.5
b = 1.5e1
c = a + b
print(c)
d = 2 ** 8
e = sin(0) + cos(0)
print(d)
print(e)
neg = -3 * -4
print(neg)
half = 7 // 2 + 7 % 2
print(half)
"""

```

```

=====
ПРОГРАМА 2
=====
17.5
256
1.0
12
4

```

Висновки

У ході виконання лабораторної роботи вивчено та розширено базовий інтерпретатор арифметичних формул, реалізований методом рекурсивного спуску.

У частинах 1–2 інтерпретатор розширено такими можливостями:

1. бінарні операції ділення на ціло `//` і остачу від ділення `%`;
2. піднесення до степеня `**` із правоасоціативністю та вищим пріоритетом, ніж у `*` `/` `//` `%`;
3. вбудовані функції `sin()` і `cos()` на основі модуля `math`;
4. унарні операції `+` і `-` перед будь-яким множником (з правильним пріоритетом відносно `**`);
5. дійсні числа у форматі фіксованої крапки (45.02) та в E-формі (1.0e-5, 28e4).

У частині 3 реалізовано операторне виконання мінімальної ~Python-мови: клас `ExprInterpreter` розширено підтримкою змінних у виразах, а клас `StmtInterpreter` забезпечує порядкове виконання операторів присвоєння та `print()` зі спільною таблицею змінних.

Усі тестові приклади підтверджують коректність реалізації: правильні пріоритети операцій, правоасоціативність степеня, коректний парсинг дійсних чисел та правильна передача значень між операторами через таблицю змінних.